PIECES & BRICKS – Demo Development Brief

Oct 27, 2026 · @Pradeep Kumar

We are building a free working demo that turns the client's 5 photos into 48×48 brick mosaics using his exact 40 colours, with a preview image and exact brick counts for each. The demo decides whether we win the full project, and the engine we build for it becomes the core of the production system.

Background

- **Client:** Khaled Maroun, launching PIECES & BRICKS, a custom brick mosaic business. Reference product: Brick.me.
- **Full project (later):** a customer website (upload photo, choose size, convert, preview, edit, see brick counts, save, order) plus a private admin panel (customers, orders, colours, inventory, sizes, pricing).
- **Client's top priority:** photo clarity, especially faces, eyes and fine details. The mosaic should look as close to the original as the 40 colours allow.
- **What the demo must prove:** our algorithm gives clear, realistic portraits on his real palette, and the brick counts are exact.
- **Meetings:** the client is available after 9:00 PM Saudi time (11:30 PM IST).

Demo scope

Phase 1 is the result set we send to the client. Phase 2 is a simple web page he can test himself, built only after Phase 1 is approved internally.

In scope, Phase 1 (offline results):

- Python conversion engine: photo in, 48×48 colour matrix out
- Processing the 5 client photos at 48×48, plus a larger size for the group photos
- For each result: a brick-style preview, a numbered grid, a brick count table, and the matrix as JSON
- With and without dithering for each photo

In scope, Phase 2 (testable demo page):

- Upload photo, crop/zoom, choose size, generate
- Show the grid, click to change a brick's colour, live brick counts

- Hosted on a private link for the client

Out of scope for the demo:

- Accounts, saving projects, orders, payments, admin panel
- PDF instructions
- Production deployment, security hardening

Inputs

Grid size

Build for a grid of **48 × 48 = 2,304 brick positions**. This is our working assumption; the client has been asked to confirm what "2×2 bricks on a 48×48 board" means. Keep the size a parameter (`--size 48`), never hard-coded, so we can also run 64×64 and 96×96 and switch quickly if the answer changes.

Palette: 40 colours

Source: `PhotoBricks_40_Colors_Easy_Codes.xlsx` from the client. Load it from a config file (`palette.json`), not from code. Always use the colour codes (C01–C40) in outputs.

Code	Name	HEX
C01	Black	#302E32
C02	White	#F7F0E3
C03	Light Beige	#FFD7C3
C04	Beige	#EBC0A4
C05	Dark Beige	#ECA279
C06	Peach	#DE9064
C07	Light Brown	#B37C51
C08	Brown	#AB5827
C09	Dark Brown	#73483A
C10	Reddish Brown	#523E37
C11	Neon Blue	#00B5CC
C12	Yellow	#FFD300

Code	Name	HEX
C13	Magenta	#FD637D
C14	Orange	#FF7E30
C15	Red	#CD2928
C16	Light Orange	#FFAE11
C17	Medium Blue	#58A3DA
C18	Blue	#0065B2
C19	Light Yellow	#FFF076
C20	Pink	#D85FA5
C21	Dark Blue	#23415D
C22	Lime	#9CBB27
C23	Green	#00A64D
C24	Dark Green	#1A4D3B
C25	Light Pink	#EDAAD6
C26	Dark Pink	#B72C75
C27	Lavender Purple	#C0AEDF
C28	Purple	#AE78C2
C29	Dark Purple	#8154AC
C30	Pastel Green	#749883
C31	Olive Green	#8D8F63
C32	Navy Blue	#7B91A5
C33	Light Grey	#A3A9AA
C34	Mouse Grey	#6F7176
C35	Light Blue	#C1EFE3
C36	Sky Blue	#93BCE6
C37	Red Wine	#853433
C38	Vanilla	#DFC790
C39	Bronze	#998769

Code	Name	HEX
C40	Dark Grey	#4B4C50

Note: C32 "Navy Blue" is actually a soft greyish blue, and C10 "Reddish Brown" is close to dark chocolate. Matching uses HEX only, so this does not affect results.

Photos: 5 from the client

Store the originals untouched in `input/` with the names below. Never overwrite them.

File name	Description	Shape
`p1_girl_teddy.jpg`	Girl with yellow teddy and a duck drawing, green table	Tall portrait
`p2_studio_family.jpg`	Studio family of 5, plain blue background	Square
`p3_birthday_family.jpg`	Family of 5 at a birthday table, busy decorations	Tall portrait
`p4_toddler_bench.jpg`	Toddler on a stone bench, bright background	Tall portrait
`p5_man_colour_bg.jpg`	Man in white T-shirt, pink/orange/yellow/blue background	Landscape

Conversion engine pipeline

The engine runs 8 steps in order and must be deterministic: the same photo and settings always give the same matrix. No random steps. Every setting below is a parameter with a default, so we can tune per photo.

1. **Load and fix orientation.** Read with Pillow, apply EXIF rotation (`ImageOps.exif_transpose`), convert to RGB.
2. **Face detection.** Detect faces with OpenCV (YuNet `FaceDetectorYN` preferred; MediaPipe as a fallback). Save face boxes; they drive the crop and the face mask.
3. **Square crop.** Default: a square around all detected faces with padding (head and shoulders for single faces). Also support a manual crop `(x, y, size)` per photo. The mosaic is always square.
4. **Upscale if small.** If the crop is under 1,000 px wide, upscale first (Real-ESRGAN or OpenCV `dnn_superres`), so enhancement works on real detail.
5. **Enhance.**

- Light denoise (`cv2.fastNlMeansDenoisingColored`, low strength)
- Local contrast with CLAHE on the L channel of LAB only, so colours don't shift
- Light unsharp mask, stronger inside the face mask (eyes, brows, mouth)
- Optional slight saturation boost for skin and clothes

6. **Simplify the background.** Outside the face/person mask, apply a bilateral or mean-shift filter, so the background uses fewer, flatter colours and doesn't steal colours from faces. Person mask: MediaPipe selfie segmentation, or a padded face box if unavailable.
7. **Downscale to the grid.** Resize to N×N with `cv2.INTER_AREA`, so each brick is the average of its area. Don't use nearest-neighbour.
8. **Colour matching in LAB.**
   - Convert the 40 palette HEX values and the N×N image to CIELAB (D65), using `skimage.color.rgb2lab` with correct sRGB handling.
   - For each cell, pick the palette colour with the smallest **CIEDE2000** distance (`skimage.color.deltaE_ciede2000`).
   - **Dithering, optional:** Floyd–Steinberg error diffusion in LAB space, with error clamping. Apply it at reduced strength inside the face mask, or not at all, since it adds noise to small faces. Test both.
   - Optional: weighted lightness, giving L* a slightly higher weight so eyes and shadows keep contrast.

Output of the engine: an N×N array of colour codes (C01–C40) plus metadata (size, settings used, palette version).

Per-photo plan

The crop decides face size, and face size decides clarity, so each photo gets its own crop plan. Expected quality, best to worst: P5, P4, P1, P2, P3.

Photo	Difficulty	Crop plan	Sizes to run	Watch for
P5 Man, colour background	Easy	Square on head and shoulders	48	Blue eyes (~1 brick); background should map cleanly to C13/C14/C12/C17

Photo	Difficulty	Crop plan	Sizes to run	Watch for
P4 Toddler on bench	Medium–easy	Tight square on face and upper body	48	Soft focus (upscale and sharpen); bright background washing out; rosy cheeks
P1 Girl with teddy	Medium	Square on her face, teddy and top of the drawing	48	Face is small in the original; strong yellow/green should not dominate
P2 Studio family	Hard	A: full family. B: tighter crop on the centre faces	48, plus 96 for version A	Faces only 6–7 bricks wide at 48; dark hair against blue
P3 Birthday family	Hardest	A: full family. B: tighter crop on the faces	48, plus 96 for version A	Busy background; simplify it hard; small faces

For every photo and size, run two variants: **no dithering** and **dithering**. That gives about 18 results in total. Pick the best per photo for the client set, but keep all of them for internal review.

Deliverables

Each result produces 5 files in `output/<photo>_<size>_<variant>/`. The client set is one PDF or image pack built from these.

File	What it is	Details
`preview.png`	Brick-style mosaic preview	Each cell drawn as a brick with a round stud (light highlight and shadow), 20–24 px per cell, thin dark gap between bricks. This is what the client judges, so it must look like real bricks.
`flat.png`	Plain pixel version	One flat colour per cell, for side-by-side comparison with the original
`grid.png`	Numbered assembly grid	Each cell shows its code number (01–40), with row/column numbers on the edges and a thicker line every 8 cells

File	What it is	Details
`counts.csv`	Brick quantities	Code, name, HEX, quantity, sorted by quantity descending; total row must equal N×N (2,304 at 48×48)
`matrix.json`	The exact matrix	`{size, palette_version, settings, grid: [["C01", …], …]}`

Comparison sheet per photo (`compare.png`): the original crop, the no-dither preview and the dither preview side by side, with the size and total bricks in the caption.

Client pack: for each photo, the best preview, the comparison sheet and the count table, plus a 1-page summary of the method in plain language.

Quality checks before sending

Nothing goes to the client until every box below is ticked. Automate the first four as tests.

- [ ] Every cell in every matrix is one of C01–C40; no other colour appears anywhere
- [ ] Counts in `counts.csv` sum to exactly N×N (2,304 at 48×48) and match a recount of `matrix.json`
- [ ] Running the same photo with the same settings twice gives an identical `matrix.json`
- [ ] `preview.png` , `grid.png` and `counts.csv` are all generated from `matrix.json` , never separately
- [ ] Eyes are visible as distinct bricks on the single-face photos (P1, P4, P5)
- [ ] Skin uses smooth skin tones (C03–C09, C38), with no stray green or grey patches on faces
- [ ] Backgrounds look flat and calm and don't pull colours away from faces
- [ ] The previews were reviewed by a second team member side by side with the original photos
- [ ] Settings used for each final result are recorded, so we can reproduce them live in a meeting

Code structure and stack

Write the engine as a clean Python package, not a notebook, so it moves straight into the FastAPI backend later.

```
pieces_bricks_demo/
  engine/
    palette.py      # load palette.json, HEX→LAB, palette version
    preprocess.py   # orientation, face detection, crop, upscale, enhance,
background
    quantize.py     # resize to N×N, CIEDE2000 matching, dithering
    render.py       # preview.png, flat.png, grid.png, compare.png
    export.py       # counts.csv, matrix.json
    pipeline.py     # run(photo, size, settings) -> result
  config/
    palette.json
    photos.yaml     # per-photo crop and settings
  input/            # originals, never modified
  output/
  tests/            # palette-only, sum, determinism tests
  run_demo.py       # CLI: python run_demo.py --photo p5 --size 48 --dither
off
```

Stack: Python 3.11+, OpenCV (`opencv-contrib-python`), Pillow, NumPy, scikit-image, MediaPipe (segmentation), optional Real-ESRGAN for upscaling. Phase 2 adds FastAPI and a React + TypeScript page with a canvas grid.

Task plan

- [ ] Set up repo, folder structure and `palette.json` from the client's Excel file
- [ ] Build `palette.py` and `quantize.py` (resize + LAB/CIEDE2000 matching), test on P5 first
- [ ] Build `render.py` and `export.py` (all 5 output files) and the automated checks
- [ ] Add preprocessing: face detection, crop, upscale, CLAHE, sharpening, background simplification
- [ ] Add Floyd–Steinberg dithering in LAB with a face-mask strength option
- [ ] Write `photos.yaml` crops and run all 18 results
- [ ] Internal review against the quality checklist; tune settings
- [ ] Build the client pack and share it with Pradeep for sending
- [ ] Phase 2, after approval: FastAPI endpoint + simple React page (upload, crop, size, generate, edit cell, live counts)

Open questions

- **Grid meaning:** does "2×2 bricks on a 48×48 board" mean 2,304 positions (our assumption) or a 48-stud plate with 576 bricks? Asked to the client; awaiting reply.

- **Physical size:** final mosaic size in cm, which sets the preview proportions.
- **Palette accuracy:** were the HEX values measured from real bricks or taken from a supplier catalogue?
- **Other sizes:** which sizes beyond 48×48 he plans to sell.